

claude-windows / 96963dab-d38f-488c-af44-530ddbefe1

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\96963dab-d38f-488c-af44-530ddbefe1\subagents\workflows\wf_dd531439-5e0\agent-a8c8073cdd3755e65.jsonl`  
- SHA-256: `3581f5c3d9dc1a5a94402cf40e789c68324941df075a186e844c632570dcda83`  
- Source modified: `2026-07-08T06:38:46+00:00`  
- Imported at: `2026-07-08T15:59:27+00:00`  
- project: `wf_dd531439-5e0`  
- session_id: `96963dab-d38f-488c-af44-530ddbefe1`
```

Transcript

1. user (2026-07-08T06:36:06.464Z)

You are reviewing an UNCOMMITTED diff in an isolated git worktree at C:/Users/avidu/AppData/Local/Temp/claude/wt524-969 (branch feat/524-l4-primary, based on origin/master).

To see the change: cd to the worktree and run `git --no-pager diff` (working tree vs HEAD) and `git --no-pager diff --stat`; read the new files docs/COMPUTE_DECOUPLED_SERVING/L4_PRIMARY_TOPOLOGY.md and tests/test_video_id_trust.py directly. Read surrounding code as needed ? do NOT limit yourself to diff hunks.

CONTEXT ? GitHub issue #524 (epic): collapse the pipeline onto the L4 worker; shrink control-plane to a thin router. This PR delivers (1) a design doc resolving the compute-topology question (recommend co-located scale-to-zero Cloud Run Jobs + bucket upload; REJECT upload-onto-L4 and persistent-GPU with a cost model), and (2) "win #2": the control-plane drain trusts the SERVER-DERIVED submit-time md5 (jobs.video_id ? submit_time_video_id, #484 ? GCS metadata md5 converted to hex by backend/storage/gcs.py::md5_hex) instead of re-hashing the fetched clip, gated by deployment.trust_submit_video_id (default True) and guarded by orchestration._looks_like_video_id (strict 32-char lowercase hex). Win #1 (validation-on-L4) is deliberately NOT in this PR ? it is delivered by the separate open branch feat/521-compile-on-L4 (phase_placement.validation); duplicating it is forbidden.

Report ONLY concrete, actionable findings with file:line. No praise, no restating the design. For each: what breaks, the exact failing input/state, and the fix. If a dimension is clean, say so explicitly.

A prior reviewer raised this finding on the SECURITY / TRUST BOUNDARY dimension:

SEVERITY: low

FILE: backend/orchestration.py:791

SUMMARY: Win #2 drops the only end-to-end check that the bytes the drain processed hash to the identity they're stored under: when trusting, there is no post-fetch recheck and the fetch is not generation-pinned, so the DB/output key rests on submit-time metadata + an unstated object-immutability assumption rather than on the code itself.

CLAIMED FAILURE: submit_time_video_id() stats the gs:// object at submit (orchestration.py:231) and persists md5Hash=X on jobs.video_id. The drain later fetches the LIVE object via acquire_local_input -> GCSStorage.fetch_to_local -> download_to_filename(dst) (gcs.py:65) with NO if_generation_match; download verifies the fetched bytes against the CURRENT object's md5Hash, not against X. If the object at that gs:// path is overwritten with different-hash content Y between the submit-time stat and the drain fetch, the trusted branch (orchestration.py:791-796) takes video_id=X with NO recheck, and this run's detection outputs + DB rows for content Y get written under key X -- silently poisoning the GLOBAL, cross-tenant content cache for future reads of X. Pre-#524 the drain re-hashed the fetched bytes (compute_video_id), so the key always matched the processed bytes and any mid-flight overwrite self-corrected. NOTE reachability: not client-triggerable in the current hosted config -- uploads are server-mediated (clients hold no direct GCS write creds), upload paths are unique uuid4 dirs (uploads/<uuid>/), and direct-URI submits are restricted to input_root -- so this is defense-in-depth today, but it becomes client-reachable the moment any future presigned direct-to-bucket upload (the doc's rejected hybrid C direction) or in-place bucket rewrite/lifecycle op

lands. (MD5-collision keying is a separate, pre-existing property of compute_video_id, unchanged by this PR.)

PROPOSED FIX: Make the trust self-verifying instead of assumption-based: carry the object generation from the submit-time stat onto the job row and fetch with if_generation_match=<gen> (fail the job on generation mismatch); or, cheaper, re-stat the object at fetch time and compare its md5 to the trusted id before taking the trusted branch (one extra metadata GET, no multi-GB re-hash). Either closes the TOCTOU window while preserving the perf win.

Adversarially VERIFY it against the real code/doc in C:/Users/avidu/AppData/Local/Temp/claude/wt524-969. Default to REFUTED unless you can construct the concrete failing input/state and confirm it in the actual code. A finding is CONFIRMED only if it is a real defect a maintainer must fix (not a style preference, not already-handled, not a misread). State the verdict and the evidence (file:line you checked).

2. assistant (2026-07-08T06:36:15.380Z)

I'll verify this finding against the actual code. Let me start by examining the relevant files.

3. user (2026-07-08T06:36:23.960Z)

warning: in the working copy of 'docs/COMPUTE_DECOUPLED_SERVING/L4_PRIMARY_TOPOLOGY.md', LF will be replaced by CRLF the next time Git touches it

warning: in the working copy of 'tests/test_video_id_trust.py', LF will be replaced by CRLF the next time Git touches it

```
backend/api/job_worker.py          | 5 +
backend/config/models.py           | 10 +
backend/orchestration.py           | 51 ++++
.../L4_PRIMARY_TOPOLOGY.md        | 225 +++++
docs/COMPUTE_DECOUPLED_SERVING/README.md | 2 +
tests/test_config_deployment.py    | 2 +
tests/test_video_id_trust.py       | 173 +++++
7 files changed, 466 insertions(+), 2 deletions(-)
```

4. user (2026-07-08T06:36:28.890Z)

```
diff --git a/backend/orchestration.py b/backend/orchestration.py
```

```
index 8b7851e..7769e4e 100644
```

```
--- a/backend/orchestration.py
```

```
+++ b/backend/orchestration.py
```

```
@@ -239,6 +239,20 @@ def submit_time_video_id(input_ref, cfg) -> Optional[str]:
     return None
```

```
+def _looks_like_video_id(value: Optional[str]) -> bool:
```

```
+    """True iff ``value`` is a well-formed ``video_id`` ? the 32-char lowercase hex MD5 that
```

```
+    ``compute_video_id`` produces (``backend/utils/hashing.py``). Guards the #524 win-#2 trust
```

```
+    path:
```

```
+    a trusted id is only reused in place of a re-hash when it has the EXACT shape the drain
```

```
+    would
```

```
+    have computed, so a malformed / truncated / wrong-encoding id can never become the DB key ?
```

```
+    it degrades to a real hash instead. ``submit_time_video_id`` already normalizes the gs://
```

```
+    base64 md5 to this hex form (#484, ``backend/storage/gcs.py::_md5_hex``)."""
```

```
+    return (
```

```
+        isinstance(value, str)
```

```
+        and len(value) == 32
```

```
+        and all(c in "0123456789abcdef" for c in value)
```

```
+    )
```

```
+    +
```

```
+    def _cached_process_payload(
```

```
+        video_id: str, validation_results, segments, requested_compute
```

```
+    ) -> Dict[str, Any]:
```

```
@@ -706,7 +720,7 @@ def _maybe_dispatch_remote(
```

```

def _execute_processing(
-   config=None, db=None, req=None, progress=None
+   config=None, db=None, req=None, progress=None, trusted_video_id=None
) -> Dict[str, Any]:
    """The full hash ? validate ? segment ? report pipeline for one video.

@@ -716,6 +730,15 @@ def _execute_processing(
    test code that calls ``_execute_processing(req)`` still works.

    ``progress`` is an optional callable(stage: str) used to surface coarse job progress.
+
+    ``trusted_video_id`` is the SERVER-DERIVED submit-time md5 the drain already knows for this
+    job (``jobs.video_id`` ? ``submit_time_video_id``, #484) ? byte-identical to
+    ``compute_video_id`` and already trusted for the submit-time cache probe. When present (and
+    ``deployment.trust_submit_video_id`` is on), it is reused so the drain SKIPS re-hashing the
+    multi-GB clip it just fetched (#524 win #2); absent / malformed ? the drain hashes as
before.
+    It is a keyword-only, internal channel ? NOT a client-supplied request field (a caller's
+    ``/api/process`` body cannot inject it; the submit hook recomputes the authoritative id).
+
    Raises on unexpected internal errors ? the caller records those as a job failure
    """
    import backend.main as _main # call-time seam: main's globals/model stay patchable
@@ -748,7 +771,31 @@ def _execute_processing(
        local_video = storage.acquire_local_input(
            req.video_path, getattr(config, "storage", None)
        )
-        video_id = _main.compute_video_id(local_video)
+        # #524 win #2: trust the SERVER-DERIVED submit-time md5 instead of re-hashing the whole
+        # (often multi-GB) clip we just fetched. The trusted id is byte-identical to
+        # compute_video_id (same bytes, same MD5 ? #484) and is ALREADY the value the submit-
time
+        # cache probe trusts, so reusing it here is consistent, not a new trust assumption.
Gated by
+        # deployment.trust_submit_video_id (default on) and only honoured when the id is well-
formed
+        # (_looks_like_video_id) ? any absence/malformation degrades to a real hash. The fetch
above
+        # is unchanged; only the redundant hash is skipped.
+        trust_submitted = bool(
+            getattr(
+                getattr(config, "deployment", None), "trust_submit_video_id", True
+            )
+        )
+        # STRICT: trust only an id in the EXACT form the drain would have produced (32-char
+        # lowercase hex, per compute_video_id / submit_time_video_id). No normalization ? a
+        # differently-cased or malformed id is a signal something is off, so re-hash rather
than
+        # silently coerce it into the DB key.
+        candidate = trusted_video_id or ""
+        if trust_submitted and _looks_like_video_id(candidate):
+            video_id = candidate
+            logger.info(
+                "[API] trusting submit-time video_id=%s ? skipped the drain re-hash (#524).",
+                video_id,
+            )
+        else:
+            video_id = _main.compute_video_id(local_video)
except Exception:
    storage.cleanup_acquired_local_input(
        req.video_path, local_video, getattr(config, "storage", None)

```

5. user (2026-07-08T06:36:37.314Z)

```

180         path = getattr(getattr(idx, "tracknet", None), "weights_path", "") or ""

```

```

181     else: # wasb_hybrid, or fusion on the (default) WASB substrate
182         path = (
183             os.environ.get("WASB_WEIGHTS")
184             or getattr(getattr(idx, "wasb", None), "weights_path", "")
185             or ""
186         )
187     return bool(path) and os.path.exists(os.path.expanduser(path))
188
189
190 def _cloud_available(cfg) -> bool:
191     """Whether THIS server can satisfy a per-request ``compute="cloud"`` dispatch (item
192 3).
193     True when the control plane is wired for Cloud Run: either the configured default
194 target is
195 already ``cloud_run`` (the operator chose cloud serving), or the Cloud Run env
196 coordinates
197 (``storage.output_root``)
198 are present so a forced per-request override can actually dispatch + collect a
199 result.
200 Surfaced in ``/api/capabilities`` so the SPA only offers the cloud toggle when it
201 would work."""
202 deployment = getattr(cfg, "deployment", None)
203 if (getattr(deployment, "compute_target", "") or "") == "cloud_run":
204     return True
205 has_job = bool(
206     os.environ.get("CLOUD_RUN_JOB") and os.environ.get("GOOGLE_CLOUD_PROJECT")
207 )
208 out_root = (getattr(getattr(cfg, "storage", None), "output_root", "") or "").strip()
209 return bool(has_job and out_root)
210
211 def _wait_progress_message(seg_name: str, elapsed_sec: float) -> str:
212     """The live job.progress line while a cloud GPU run is in flight (#482). Honest and
213 minute-granular: the control plane knows ELAPSED time, not frames ? frame-level %
214 comes
215 with the worker-side heartbeat (the #482 follow-up). Under a minute reads as
216 "starting"
217 (queue/cold-start), so the very first tick already shows movement."""
218 minutes = int(elapsed_sec) // 60
219 if minutes < 1:
220     return f"segmenting ({seg_name}) on cloud GPU ? starting up"
221 return f"segmenting ({seg_name}) on cloud GPU ? {minutes} min elapsed"
222
223 def submit_time_video_id(input_ref, cfg) -> Optional[str]:
224     """The video's md5 (the canonical ``video_id``) resolvable CHEAPLY at submit time ?
225 gs://
226 object metadata only, one stat, no bytes (#484). ``None`` ? unknown: local files
227 keep
228 deferred hashing on the drain (DEFERRED_HARDENING #16), and composite objects carry
229 no
230 ``md5Hash``. Never raises ? a metadata miss must not block a submit."""
231 if getattr(input_ref, "backend", "") != "gcs":
232     return None
233 sc = _storage_settings(cfg)
234 if not sc:
235     return None
236 try:
237     st = storage.get_storage("gcs", config=sc).stat(input_ref)
238     return getattr(st, "md5", None) or None
239 except Exception: # noqa: BLE001 ? best-effort identity, the drain still hashes
240     logger.info(

```

```

235         "[API] submit-time md5 unavailable for %s (metadata stat failed)",
236         getattr(input_ref, "uri", input_ref),
237         exc_info=True,
238     )
239     return None
240
241
242     def _looks_like_video_id(value: Optional[str]) -> bool:
243         """True iff ``value`` is a well-formed ``video_id`` ? the 32-char lowercase hex MD5
that
244         ``compute_video_id`` produces (``backend/utils/hashing.py``). Guards the #524 win-#2
trust path:
245         a trusted id is only reused in place of a re-hash when it has the EXACT shape the
drain would
246         have computed, so a malformed / truncated / wrong-encoding id can never become the
DB key ?
247         it degrades to a real hash instead. ``submit_time_video_id`` already normalizes the
gs://
248         base64 md5 to this hex form (#484, ``backend/storage/gcs.py::_md5_hex``)."""
249         return (
250             isinstance(value, str)
251             and len(value) == 32
252             and all(c in "0123456789abcdef" for c in value)
253         )
254
255
256     def _cached_process_payload(
257         video_id: str, validation_results, segments, requested_compute
258     ) -> Dict[str, Any]:
259         """The /api/process result for a video whose detection already exists ? ONE shape
for
260         both producers (the drain's DB-cache short-circuit and the submit-time cache,
#484)."""
261         return {
262             "video_id": video_id,
263             "status": "success",
264             "message": (
265                 f"Reused {len(segments)} cached play segments. "
266                 "Pass force=true to reprocess this video."
267             ),
268             "results": validation_results or [],
269             "play_segments": segments,
270             "cached": True,
271             "compute": {"path": "cached", "requested": requested_compute},
272         }
273
274
275     def _read_bucket_json(uri: str, storage_config: dict) -> Optional[Dict[str, Any]]:
276         """Fetch + parse one small JSON object (cloud backends require a real dst ? the
277         ``_ingest_remote_segments`` pattern). Raises on fetch/parse errors; callers
decide."""
278         ref = storage.parse_uri(uri)
279         with tempfile.TemporaryDirectory(prefix="cr-report-") as td:
280             local = storage.fetch(
281                 uri, os.path.join(td, ref.name or "report.json"), config=storage_config
282             )
283             with open(local, encoding="utf-8") as f:
284                 loaded = json.load(f)
285             return loaded if isinstance(loaded, dict) else None
286
287
288     # One re-hydrate at a time per video_id (#492 review blocker 1): two concurrent cache
hits
289     # from an empty DB must not both ingest intervals.csv ? the loser waits, then finds the
290     # winner's rows in the DB and reuses them. Process-wide is enough: the CP is single-

```

```

instance
291     # (maxScale=1, spec D8) and SQLite serializes cross-process writers anyway.
292     _REHYDRATE_LOCKS: Dict[str, threading.Lock] = {}
293     _REHYDRATE_LOCKS_GUARD = threading.Lock()
294
295
296     def _rehydrate_lock(video_id: str) -> threading.Lock:
297         with _REHYDRATE_LOCKS_GUARD:
298             return _REHYDRATE_LOCKS.setdefault(video_id, threading.Lock())
299

```

6. user (2026-07-08T06:36:37.763Z)

```

1     """Google Cloud Storage backend ? first-class object buckets.
2
3     Uses ``google-cloud-storage``, imported lazily so ``import backend.storage`` never pulls
it;
4     install with the ``storage-gcs`` extra. Credentials come from Application Default
Credentials
5     (``GOOGLE_APPLICATION_CREDENTIALS`` pointing at a service-account JSON, or workload
identity) ?
6     never from config. A ``client`` may be injected for tests.
7     """
8
9     from __future__ import annotations
10
11     import base64
12     import binascii
13     import logging
14     from datetime import timedelta
15     from importlib import import_module
16     from typing import Any, Iterator, Optional
17
18     from backend.storage.base import StorageBackend
19     from backend.storage.ref import StorageRef, StorageStat
20
21     logger = logging.getLogger(__name__)
22
23
24     def _md5_hex(md5_b64: "str | None") -> Optional[str]:
25         """GCS reports ``md5Hash`` as base64 of the raw digest; ``compute_video_id`` (the DB
key)
26         is a HEX digest ? convert so the two are directly comparable (#484). Composite
objects
27         have no md5Hash ? None; a malformed value is treated as absent, never raised."""
28         if not md5_b64:
29             return None
30         try:
31             raw = base64.b64decode(md5_b64, validate=True)
32         except (binascii.Error, ValueError):
33             return None
34         return raw.hex() if len(raw) == 16 else None
35
36
37     class GCSStorage(StorageBackend):
38         def __init__(
39             self,
40             *,
41             client: Any = None,
42             project: Optional[str] = None,
43             signed_url_ttl: int = 3600,
44             **_ignored,
45         ) -> None:
46             self._ttl = signed_url_ttl
47             if client is not None:

```

```

48         self._client = client
49         return
50     storage_module: Any = import_module("google.cloud.storage")
51     self._client = storage_module.Client(project=project)
52
53     def _blob(self, ref: StorageRef):
54         return self._client.bucket(ref.bucket).blob(ref.key)
55
56     def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
57         if dst is None:
58             raise ValueError("GCSStorage.fetch_to_local requires a dst path")
59         import os
60
61         dst = str(dst)
62         if os.path.exists(dst) and not overwrite:
63             return dst
64         os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
65         self._blob(ref).download_to_filename(dst)
66         return dst
67
68     def put(self, local_path, ref, *, content_type=None) -> StorageRef:
69         self._blob(ref).upload_from_filename(str(local_path), content_type=content_type)
70         return ref
71
72     def exists(self, ref) -> bool:
73         return bool(self._blob(ref).exists())
74
75     def stat(self, ref) -> StorageStat:
76         b = self._blob(ref)
77         b.reload()
78         return StorageStat(
79             size=int(b.size or 0),
80             modified=b.updated,
81             etag=b.etag,
82             content_type=b.content_type,
83             md5=_md5_hex(getattr(b, "md5_hash", None)),
84         )
85
86     def list(self, prefix) -> Iterator[StorageRef]:
87         for blob in self._client.list_blobs(prefix.bucket, prefix=prefix.key):
88             yield StorageRef(
89                 "gcs", prefix.bucket, blob.name, f"gcs://{prefix.bucket}/{blob.name}"
90             )
91
92     def delete(self, ref) -> None:
93         self._blob(ref).delete()
94
95     def open_writer(self, ref, *, content_type=None):
96         blob = self._blob(ref)
97         if content_type:
98             blob.content_type = content_type
99         # ignore_flush: BlobWriter only accepts flush() on chunk boundaries and raises
100         # otherwise; upload staging writes sequentially and finalizes with close(), so a
101         # caller-side flush must be a no-op, not an error. Bytes stream out per internal
102         # chunk (default 40 MiB) ? the whole object is never held in memory.
103         return blob.open("wb", ignore_flush=True)
104
105     def signed_url(self, ref, *, ttl=None, method="GET") -> str:
106         blob = self._blob(ref)
107         exp = timedelta(seconds=ttl or self._ttl)
108         creds = getattr(self._client, "_credentials", None)
109         # Local-key signing (a service-account JSON key exposes a `signer`) works out of
110         # the box. But
111         # on Cloud Run / GCE the ADC credentials come from the metadata server and have
112         # NO private key,

```

```

111         # so a plain generate_signed_url raises "you need a private key". Fall back to
the IAM signBlob
112         # API ? which requires the runtime SA to hold
roles/iam.serviceAccountTokenCreator on ITSELF.
113         # ? signBlob canNOT ride the storage client's own token (#475, found on the
first real-creds
114         # run): that token is scoped to devstorage.* only (Client.SCOPE), which
iamcredentials rejects
115         # with ACCESS_TOKEN_SCOPE_INSUFFICIENT ? and a metadata credential may still
report the
116         # "default" email alias. Mint a separate cloud-platform-scoped credential
instead: its refresh
117         # both authorizes signBlob and resolves the real SA email. Failures are LOGGED
before deferring
118         # to the default call, so a broken sign path can't masquerade as a missing-key
error.
119         if creds is not None and not hasattr(creds, "signer"):
120             try:
121                 sa_email, token = self._signblob_identity()
122                 if sa_email:
123                     return blob.generate_signed_url(
124                         version="v4",
125                         expiration=exp,
126                         method=method,
127                         service_account_email=sa_email,
128                         access_token=token,
129                     )
130                 logger.warning(
131                     "[gcs] keyless signing skipped: runtime SA email unresolved (got
132                     %r); "
133                     "falling back to key-based signing",
134                     sa_email,
135                 )
136             except Exception as e:
137                 logger.warning(
138                     "[gcs] keyless signBlob path failed (%s); falling back to key-based
139                     signing",
140                     e,
141                 )
142             return blob.generate_signed_url(version="v4", expiration=exp, method=method)
143
144     @staticmethod
145     def _signblob_identity() -> "tuple[Optional[str], Optional[str]]":
146         """(sa_email, access_token) for the IAM signBlob call ? a FRESH cloud-platform-
147         scoped
148         credential from ADC, never the storage client's devstorage-scoped token (#475).
149         Isolated
150         as a seam so tests can fake the metadata-server identity without real
151         credentials."""
152         import google.auth
153         from google.auth.transport.requests import Request as _AuthRequest
154
155         sign_creds, _ = google.auth.default(
156             scopes=["https://www.googleapis.com/auth/cloud-platform"]
157         )
158         sign_creds.refresh(_AuthRequest())
159         sa_email = getattr(sign_creds, "service_account_email", None)
160         if not sa_email or sa_email == "default":
161             return None, None
162         return sa_email, sign_creds.token

```

7. user (2026-07-08T06:36:44.097Z)

```

1         """Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md

```

```

P3).
2
3     Extracted verbatim from backend.main to shrink the orchestrator; behavior is unchanged.
4     The worker DRAINS the jobs table via `claim_due_job` and honours WAIT_AND_RESUME: a job
5     that raises `JobWaiting` is parked (status 'waiting' + `next_poll_at` + progress) and
6     re-claimed when due ? with no central scheduler (the DB is the clock).
7
8     The monkeypatch contract these functions rely on (and that tests depend on):
9     every cross-function seam ? `db_instance`, `global_config`, `_execute_processing`,
10    `_arm_wake_timer`, `_get_executor`, `_MAX_DRAIN_WAKE_SEC` ? is resolved THROUGH the
11    `backend.main` module object at call-time (`import backend.main as _main`), never as a
12    bare local. `backend.main` re-exports these names, so `monkeypatch.setattr(backend.main,
13    ...)` still intercepts the inter-function calls (e.g. a parked job arms the *patched*
14    `_arm_wake_timer`, so no real daemon timer leaks in tests). Importing `backend.main`
lazily
15    inside the functions also avoids the circular import at module load (main.py imports
this
16    module; this module must not import main at import time).
17    """
18
19    import json
20    import logging
21    import os
22    import threading
23    import traceback
24    from concurrent.futures import ThreadPoolExecutor
25    from typing import Any, Dict, Optional
26
27    logger = logging.getLogger(__name__)
28
29    # Async job executor (DEFERRED_HARDENING_PLAN #16). max_workers=1 by default: a single
30    # self-hosted box serializes GPU work, and the Gemini provider is rate-fragile under
31    # parallel uploads (2026-06-10 ops learning: serial uploads + backoff). The segmenters
32    # already parallelize their AI handoff internally via their own pools.
33    # EXCEPTION (#466): when the server CAN dispatch to Cloud Run (see
_cloud_dispatch_possible), a REMOTE
34    # job is only a bucket-poll (I/O-bound; GPU is on ephemeral L4s, WASB-only has no
Gemini), so the drain
35    # scales to deployment.max_concurrent_remote_jobs to overlap those. This does NOT
parallelize local
36    # work: CPU/GPU-heavy in-process detection + the compile stitch serialize via
37    # orchestration._LOCAL_COMPUTE_LANE regardless of the worker count, so the single-box
invariant holds
38    # and compile/local-override jobs stay serial. The count is fixed at first
_get_executor() creation.
39    #
40    # ? O2 RISK (CODE_CLEANUP_AUDIT $3c): a single worker means ONE stuck or hung job
41    # (e.g. hung Gemini generate, hung WSL GPU call, hung Cloud Run poll) blocks ALL
42    # subsequent jobs ? they stay 'queued' forever. Mitigations already in place:
43    #   - ExecutionPolicy op_timeout_sec kills hung generate/processing calls
44    #   - detector timeout_sec kills hung WSL/GPU calls
45    #   - Cloud Run poll has a max_wait_sec bound
46    #   - ffmpeg/ffprobe subprocesses use bounded media timeouts
47    # A future slice should add a per-job wall-clock timeout (the executor itself
48    # cannot enforce timeouts on the Thread).
49    _job_executor: Optional[ThreadPoolExecutor] = None
50    # Drain worker count, fixed at first _get_executor() creation. 1 (serial) by default;
configure_drain()
51    # raises it for cloud_run (#466). A module global (not a _get_executor arg) so the many
call sites +
52    # the tests that monkeypatch `_get_executor` as a no-arg lambda are unchanged.
53    _drain_max_workers: int = 1
54
55
56    def _cloud_dispatch_possible(config: Any = None) -> bool:

```

```

57         """True if this server CAN dispatch a job to Cloud Run ? either the profile sets
58         ``deployment.compute_target=cloud_run``, OR the per-request ``compute='cloud'``
override path is
59         wired (``CLOUD_RUN_JOB`` + ``GOOGLE_CLOUD_PROJECT`` env + an output bucket). Mirrors
60         orchestration._cloud_available so drain concurrency tracks the ACTUAL remote-
dispatch capability,
61         not just the server default (so compute='cloud' overrides on a local-default server
parallelize too)."""
62         dep = getattr(config, "deployment", None)
63         if getattr(dep, "compute_target", "") == "cloud_run":
64             return True
65         if os.environ.get("CLOUD_RUN_JOB") and os.environ.get("GOOGLE_CLOUD_PROJECT"):
66             stor = getattr(config, "storage", None)
67             if (getattr(stor, "output_root", "") or "").strip():
68                 return True
69         return False
70
71
72     def _resolve_drain_workers(config: Any = None) -> int:
73         """Drain concurrency (#466): a server that CAN dispatch to Cloud Run (see
_cloud_dispatch_possible)
74         may run ``deployment.max_concurrent_remote_jobs`` drain workers so I/O-bound remote
dispatch/poll
75         OVERLAPS. This does NOT parallelize CPU/GPU-heavy LOCAL work ? in-process detection
and the compile
76         stitch serialize via orchestration._LOCAL_COMPUTE_LANE regardless of the worker
count ? so the
77         single-box GPU + rate-fragile Gemini invariant holds, and compile/local-override
jobs stay serial.
78         A server with no remote capability stays serial (1). Defaults to 1 when config is
absent
79         (tests / non-server) ? behaviour unchanged."""
80         if _cloud_dispatch_possible(config):
81             dep = getattr(config, "deployment", None)
82             return max(1, int(getattr(dep, "max_concurrent_remote_jobs", 1) or 1))
83         return 1
84
85
86     def configure_drain(config: Any = None) -> None:
87         """Set the drain worker count from ``config`` BEFORE the executor is created (#466).
Call once at
88         server startup. No-op effect once the executor already exists (the count is fixed at
creation)."""
89         global _drain_max_workers
90         _drain_max_workers = _resolve_drain_workers(config)
91
92
93     def _get_executor() -> ThreadPoolExecutor:
94         """Lazy module-global executor; tests monkeypatch this for inline execution. Worker
count is
95         ``_drain_max_workers`` (1 by default; raised for cloud_run via ``configure_drain``,
#466)."""
96         global _job_executor
97         if _job_executor is None:
98             _job_executor = ThreadPoolExecutor(
99                 max_workers=_drain_max_workers, thread_name_prefix="jobworker"
100             )
101         return _job_executor
102
103
104     class JobWaiting(Exception):
105         """Raised inside the pipeline to SELF-SCHEDULE a job (EXECUTION_POLICY.md P3,
WAIT_AND_RESUME). It means "I can't finish now ? park me and resume in `delay_sec``,
106         NOT a failure. The worker persists `next_poll_at` + `progress` (via
`set_job_waiting`),

```

```

108         releases the thread, and re-claims the job when due (`claim_due_job`). No work to
date
109         is the segmenter's `_execute_processing`, run UNDER the destroy-after-confirm
contract.
110
111         The wiring is additive: the production segmenter path never raises this today (zero
112         behaviour change), so a job runs exactly as before. The hook is what a later slice
113         (resumable AI handoff) raises once a long op exhausts its in-process wait budget.
114         """
115
116         def __init__(self, delay_sec: float, progress: Optional[str] = None) -> None:
117             super().__init__(f"job parked for {delay_sec:.0f}s")
118             self.delay_sec = max(0.0, float(delay_sec))
119             self.progress = progress
120
121
122     def _job_to_request(job: Dict[str, Any]):
123         """Reconstruct the submit-time ProcessRequest from a claimed job row, so a job
124         re-claimed by `claim_due_job` (queued OR resumed-from-waiting) runs identically to
125         the original submit. The row stores exactly the ProcessRequest fields."""
126         import backend.main as _main
127
128         raw_params = job.get("params")
129         params = (
130             raw_params if isinstance(raw_params, dict) else json.loads(raw_params or "{}")
131         )
132         return _main.ProcessRequest(
133             video_path=job["video_path"],
134             player_pool=job.get("player_pool"),
135             max_frames=job.get("max_frames"),
136             segmenter=job.get("segmenter"),
137             # v8 compute-picker: the worker runs _execute_processing ?
138             # honours this, so it MUST survive submit?reconstruct (unlike locale, which is
only read off
139             # the job row for analytics). NULL ? server default = pre-picker behaviour.
140             compute=job.get("compute"),
141             force=bool(params.get("force")),
142         )
143
144
145     def _run_claimed_job(job: Dict[str, Any], config: Any, db: Any) -> None:
146         """Run ONE job already claimed (`status='running'`) by `claim_due_job`, recording
its
147         terminal state ? OR parking it for resume on `JobWaiting` (WAIT_AND_RESUME).
148
149         Job `status` is the EXECUTION state (queued|waiting|running|done|failed): 'done'
means
150         the worker finished ? the pipeline's own verdict (validation failure etc.) lives in
151         result["status"]; 'failed' means the worker itself crashed/raised; 'waiting' means
the
152         job self-scheduled and will be re-claimed when `next_poll_at` is due.
153         """
154         import backend.main as _main
155
156         job_id = job["id"]
157         # Dispatch by kind: 'compile' (#329, the async reel stitch) vs 'process'
(NULL/default ? the
158         # original /api/process pipeline). Both record their terminal state the same way;
only process
159         # jobs feed the M8 cost ledger + M11 flywheel (a compile has no detection
usage/segments).
160         is_compile = job.get("kind") == "compile"
161         try:
162             if is_compile:

```

```

163         result = _main._execute_compile(
164             config,
165             db,
166             job,
167             progress=lambda stage: db.set_job_progress(job_id, stage),
168         )
169     else:
170         req = _main._job_to_request(job)
171         result = _main._execute_processing(
172             config,
173             db,
174             req,
175             progress=lambda stage: db.set_job_progress(job_id, stage),
176             # #524 win #2: hand the drain the submit-time md5 already on the job row
177             # (``submit_time_video_id``, #484 ? server-derived, byte-identical to
178             # compute_video_id) so it can skip re-hashing the fetched clip. "" ?
unknown
179             # (local upload / non-gcs URI) ? the drain hashes as before.
180             trusted_video_id=job.get("video_id") or None,
181         )
182     if result.get("video_id"):
183         db.set_job_video_id(job_id, result["video_id"])
184     db.mark_job_done(job_id, result)
185     if not is_compile:
186         _maybe_record_job_cost(
187             job, result, config, db
188         ) # M8 cost ledger (default-OFF; process jobs only)
189         _maybe_run_flywheel(
190             job, result, config
191         ) # M11 data-flywheel (process jobs only)
192 except _main.JobWaiting as w:
193     # Cooperative re-enqueue: persist progress + next_poll_at, release the worker.
194     logger.info(f"Job {job_id} parked for {w.delay_sec:.0f}s (resume when due).")
195     db.set_job_waiting(job_id, w.delay_sec, progress=w.progress)
196 except Exception as e:
197     logger.error(f"Job {job_id} crashed: {e}\n{traceback.format_exc()}")
198     db.mark_job_failed(job_id, f"{type(e).__name__}: {e}")
199
200
201 def _maybe_record_job_cost(
202     job: Dict[str, Any], result: Dict[str, Any], config: Any, db: Any
203 ) -> None:
204     """M8 cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8). DEFAULT-OFF: a no-op unless
205     ``config.billing.enabled``. When on, compute the job's cost from ``result['usage']``
206     (= {gpu_seconds, wall_seconds, egress_gb, gemini_usd, gpu_type}) × the versioned
pricebook, and
207     persist it.
208
209     ? **Honest gap (M8 scope):** the pipeline does **not** emit ``result['usage']`` yet,
so even with
210     ``billing.enabled=True`` AND a real calibrated pricebook, every recorded cost is
**0** until a
211     FOLLOW-UP wires the worker to emit usage (gpu/wall/egress/gemini). M8 lands the
ledger + pricebook
212     + the recording seam; the usage-emission step is separate (the metering hooks on the
real run).
213     With the placeholder pricebook the cost is 0 regardless.
214
215     Best-effort ? never fails a completed job because cost bookkeeping hiccuped. ``job``
is the
216     claimed row (carries ``owner_id``); ``result`` is the pipeline output."""
217
218     billing_cfg = getattr(config, "billing", None)
219     if not billing_cfg or not getattr(billing_cfg, "enabled", False):
220         return

```

```

221     try:
222         usage = (result or {}).get("usage") or {}
223         db.record_job_cost(
224             job["id"],
225             usage=usage,
226             pricebook_version=billing_cfg.pricebook_version,
227             gpu_type=usage.get("gpu_type"),
228             compute_backend=(result or {}).get("compute_backend"),
229             owner_id=job.get("owner_id"),
230             margin=billing_cfg.margin,
231         )
232         logger.info(
233             "Recorded cost for job %s (pricebook=%s).",
234             job["id"],
235             billing_cfg.pricebook_version,
236         )
237     except Exception as e: # never wedge a completed job on bookkeeping
238         logger.warning(
239             "Cost recording for job %s failed (non-fatal): %s", job.get("id"), e
240         )
241
242
243     def _maybe_run_flywheel(
244         job: Dict[str, Any], result: Dict[str, Any], config: Any
245     ) -> None:
246         """M11 data-flywheel (COMPUTE_DECOUPLED_SERVING M11, D12). DEFAULT-OFF + INERT: a
no-op unless
247         ``config.flywheel.enabled`` AND attested consent ? and consent is a **faked hard-
deny** today
248         (``flywheel.consent_attested`` returns False for every job until counsel + the
consent schema
249         land), so this captures **nothing** in production. When activated it captures the
consented job's
250         artifacts into the per-video workspace (? later labeling ? the golden training set).
251
252         Best-effort ? never fails a completed job because the flywheel hiccuped. ``job`` is
the claimed
253         row (carries ``owner_id``); ``result`` is the pipeline output."""
254
255         flywheel_cfg = getattr(config, "flywheel", None)
256         if not flywheel_cfg or not getattr(flywheel_cfg, "enabled", False):
257             return
258         try:
259             from backend import flywheel
260
261             flywheel.run_for_job(job, result, config)
262         except Exception as e: # never wedge a completed job on the flywheel
263             logger.warning("Flywheel for job %s failed (non-fatal): %s", job.get("id"), e)
264
265
266     def _drain_due_jobs(config: Any, db: Any) -> int:
267         """One worker tick: claim and run every currently-runnable job (queued, or waiting
whose
268         `next_poll_at` is due), earliest-due first, until the table has nothing runnable.
269
270         Each parked job (`JobWaiting` ? status 'waiting') drops out of this drain and is
picked
271         up by a FUTURE drain once due; we schedule that follow-up drain via
`_schedule_drain` so
272         a single-worker box keeps making progress with no central timer (the DB is the
clock).
273
274         Returns the number of jobs that reached a terminal/parked state this tick.
275         """
276         import backend.main as _main

```

```

277     ran = 0
278     while True:
279         job = db.claim_due_job()
280         if job is None:
281             break
282         ran += 1
283         _main._run_claimed_job(job, config, db)
284     # If anything is parked-but-not-yet-due, arrange a wake-up so it resumes without a
new
285     # submit. On the single-box executor this is a delayed re-drain; multi-worker just
relies
286     # on the next submit/drain. Best-effort ? never raises into the worker.
287     _main._schedule_next_drain_if_waiting(config, db)
288     return ran
289
290
291 def _schedule_next_drain_if_waiting(config: Any, db: Any) -> None:
292     """If any job is parked in 'waiting', arm a re-drain for when the soonest one is
due, so a
293     self-scheduled job resumes with no further client submit. The drain itself re-checks
294     due-ness, so an early wake is harmless and a clamped wake for a far-future job is
fine."""
295     import backend.main as _main
296
297     try:
298         delay = db.seconds_until_next_due()
299     except Exception as e: # pragma: no cover - defensive; never wedge the worker
300         logger.error(f"Could not compute next-due delay: {e}")
301         return
302     if delay is None:
303         return # nothing waiting
304     # Clamp so a far-future job doesn't pin a thread forever; the drain re-checks due-
ness.
305     _main._arm_wake_timer(max(0.0, min(delay, _main._MAX_DRAIN_WAKE_SEC)), config, db)
306
307
308 def _arm_wake_timer(delay_sec: float, config: Any, db: Any) -> None:
309     """Arm a single daemon timer that re-submits a drain after ``delay_sec`` (no central
310     scheduler ? the DB is the clock). Isolated as the sole timer-creating seam so tests
can
311     monkeypatch it to a no-op and assert the *scheduling decision* without real
waiting."""
312     import backend.main as _main
313
314     timer = threading.Timer(
315         delay_sec,
316         lambda: _main._get_executor().submit(_main._drain_due_jobs, config, db),
317     )
318     timer.daemon = True
319     timer.start()
320
321
322     #: A parked job further out than this is re-checked by a capped wake rather than one
long
323     #: sleeping timer ? keeps the wake cadence bounded without a central scheduler.
324     _MAX_DRAIN_WAKE_SEC = 60.0
325
326
327     #
-----
328     # Restart-safe remote jobs (CONTROL_PLANE_DURABLE_REBUILD P3, #471) ? the 2026-07-05
loss shape:
329     # a running cloud detection's done-marker poll lives only in this process's memory, so a
restart
330     # orphaned the job while the L4 worker kept computing; its 29 rallies landed in the

```

```

bucket with
331     # no listener. With the md5 persisted at submit (P4), outputs/<md5>/report.json IS a
durable
332     # done-signal the restarted control plane can re-arm on ? completing the job WITHOUT
ever
333     # re-dispatching (no double GPU spend).
334
335     #: Resume-poll cadence. Module-level so tests drive the loop with a zero-wait cadence.
336     _RESUME_POLL_EVERY_SEC = 15.0
337
338
339     def resume_interrupted_remote_jobs(config: Any, db: Any) -> int:
340         """Re-arm the durable done-signal poll for md5-keyed RUNNING process jobs a restart
341         orphaned (the rows ``recover_stale_jobs`` deliberately left alone). Call once at
startup,
342         after ``recover_stale_jobs`` and before any drain. Each job resumes on its own
daemon
343         thread (bounded by the handful of rows a single-instance CP can have in flight);
rows are
344         NEVER re-dispatched from here. Returns the number of resumes started.
345
346         On a box that cannot dispatch to Cloud Run at all (the appliance), no remote worker
can
347         exist ? the rows get today's honest terminal failure instead."""
348         rows = db.interrupted_remote_process_jobs()
349         if not rows:
350             return 0
351         if not _cloud_dispatch_possible(config):
352             for job in rows:
353                 db.mark_job_failed(str(job["id"]), "interrupted by server restart")
354                 logger.warning(
355                     "[JOBS] %d interrupted md5-keyed process job(s) failed: this server cannot "
356                     "dispatch to Cloud Run, so no remote worker can be in flight.",
357                     len(rows),
358                 )
359             return 0
360         # Prove each row was actually CLOUD-dispatched (#494 review round 2): an explicit
361         # compute='cloud' always was; NULL means "server default", which is remote ONLY when
the
362         # default target itself is cloud_run (on a default-local server with cloud-override
363         # capability, a NULL row ran in-process ? no remote worker exists to resume).
364         default_is_cloud = (
365             getattr(getattr(config, "deployment", None), "compute_target", "") ==
"cloud_run"
366         )
367         started = 0
368         for job in rows:
369             # Same normalization the execution path applies (_maybe_dispatch_remote
lowercases
370             # req.compute) ? "LOCAL"/" cloud " must mean at recovery exactly what they meant
at run.
371             requested = (str(job.get("compute") or "")).strip().lower()
372             if requested != "cloud" and not default_is_cloud:
373                 db.mark_job_failed(str(job["id"]), "interrupted by server restart")
374                 continue
375             t = threading.Thread(
376                 target=_resume_one_remote_job,
377                 args=(config, db, dict(job)),
378                 daemon=True,
379                 name=f"resume-{str(job['id'])[:8]}",
380             )
381             t.start()
382             started += 1
383         if started:
384             logger.warning(

```

```

385         "[JOBS] re-armed the durable done-signal poll for %d interrupted remote
job(s).",
386         started,
387     )
388     return started
389
390
391 def _resume_one_remote_job(config: Any, db: Any, job: Dict[str, Any]) -> None:
392     """One orphaned job's resume: poll ``outputs/<md5>/report.json`` (bounded by the
same
393     ``deployment.remote_poll_max_wait_sec`` budget a live dispatch gets), then complete
or
394     fail the job HONESTLY from what the bucket says. Never dispatches."""
395     from backend import orchestration, storage
396     from backend.infrastructure.execution_policy import (
397         ExecutionPolicy,
398         PolicyTimeout,
399         poll_until,
400     )
401
402     job_id = str(job["id"])
403     video_id = str(job["video_id"])
404     try:
405         raw_params = job.get("params")
406         params = (
407             raw_params
408             if isinstance(raw_params, dict)
409             else json.loads(raw_params or "{}")
410         )
411         if params.get("force"):
412             # Defense-in-depth (#494 review): the SQL carve-out already excludes forced
rows,
413             # but a forced job must NEVER complete from the bucket cache ? force
deliberately
414             # bypasses it, and nothing proves the report belongs to the forced run
rather
415             # than an older completed one.
416             db.mark_job_failed(
417                 job_id,
418                 "control plane restarted during a forced reprocess; the cached result "
419                 "cannot be trusted for force=true ? re-submit with force",
420             )
421             return
422         if (str(job.get("compute") or "").strip().lower() == "local":
423             # Defense-in-depth (#494 review round 2): an explicit local run has no
remote
424             # worker; a same-md5 bucket report would be an OLDER run's output, not this
job's.
425             db.mark_job_failed(job_id, "interrupted by server restart")
426             return
427         sc = orchestration._storage_settings(config)
428         output_root = ((sc or {}).get("output_root") or "").rstrip("/")
429         if not sc or not output_root:
430             db.mark_job_failed(
431                 job_id,
432                 "interrupted by server restart (no output bucket configured to recover
from)",
433             )
434             return
435         report_uri = f"{output_root}/outputs/{video_id}/report.json"
436         max_wait = float(
437             getattr(getattr(config, "deployment", None), "remote_poll_max_wait_sec",
3900.0)
438             or 3900.0
439         )

```

```

440     policy = ExecutionPolicy(
441         poll_every_n_sec=_RESUME_POLL_EVERY_SEC,
442         op_timeout_sec=120.0,
443         max_wait_sec=max_wait,
444     )
445     db.set_job_progress(job_id, "recovering: waiting for the in-flight cloud run")
446     try:
447         poll_until(
448             lambda: True if storage.exists(report_uri, config=sc) else None,
449             policy,
450             label=f"resume-{job_id[:8]}",
451             logger=logger,
452             # The #493 live heartbeat: job.progress moves once per still-waiting
tick.
453             on_tick=lambda elapsed: db.set_job_progress(
454                 job_id,
455                 "recovering: waiting for the in-flight cloud run ? "
456                 f"{int(elapsed) // 60} min",
457             ),
458         )
459     except PolicyTimeout:
460         db.mark_job_failed(
461             job_id,
462             "control plane restarted mid-run and the cloud worker's result never "
463             "appeared within the poll budget; re-submit the video (a finished result
"
464             "re-hydrates instantly)",
465         )
466         return
467     payload = orchestration.cached_process_result(
468         db, config, video_id, str(job.get("video_path") or "")
469     )
470     if payload is not None:
471         db.mark_job_done(job_id, payload)
472         logger.warning(
473             "[JOBS] resumed job %s: ingested the in-flight worker's result for %s.",
474             job_id,
475             video_id,
476         )
477         return
478     # The report exists but did not realize into a usable result ? surface its
verdict.
479     status = "unreadable"
480     try:
481         report = orchestration._read_bucket_json(report_uri, sc) or {}
482         status = str(report.get("status") or "unreadable")
483     except Exception: # noqa: BLE001 ? the failure message is best-effort
484         pass
485     db.mark_job_failed(
486         job_id,
487         f"cloud worker finished without a usable result after a control-plane
restart "
488         f"(report status: {status}); re-submit the video",
489     )
490     except Exception as e: # noqa: BLE001 ? a resume crash must land as a job failure,
not vanish
491         logger.error("[JOBS] resume of job %s crashed: %s", job_id, e, exc_info=True)
492     try:
493         db.mark_job_failed(job_id, f"restart recovery failed: {e}")
494     except Exception: # noqa: BLE001
495         pass
496

```

```

diff --git a/backend/api/job_worker.py b/backend/api/job_worker.py
index a90b14e..f8da26b 100644
--- a/backend/api/job_worker.py
+++ b/backend/api/job_worker.py
@@ -173,6 +173,11 @@ def _run_claimed_job(job: Dict[str, Any], config: Any, db: Any) -> None:
     db,
     req,
     progress=lambda stage: db.set_job_progress(job_id, stage),
+    # #524 win #2: hand the drain the submit-time md5 already on the job row
+    # (`submit_time_video_id`, #484 ? server-derived, byte-identical to
+    # compute_video_id) so it can skip re-hashing the fetched clip. "" ? unknown
+    # (local upload / non-gcs URI) ? the drain hashes as before.
+    trusted_video_id=job.get("video_id") or None,
)
    if result.get("video_id"):
        db.set_job_video_id(job_id, result["video_id"])
diff --git a/backend/config/models.py b/backend/config/models.py
index a7c997c..71e43b8 100644
--- a/backend/config/models.py
+++ b/backend/config/models.py
@@ -481,6 +481,16 @@ class DeploymentConfig(BaseModel):
    # 1800s ExecutionPolicy default undercut real 1080p60 jobs (~22 min detect + queue/cold-
start,
    # observed on the 2026-07-05 owner CUJ) and could abandon a slow-but-healthy paid GPU run.
    remote_poll_max_wait_sec: float = Field(default=3900.0, gt=0)
+    # Trust the SUBMIT-TIME md5 instead of re-hashing the fetched file on the drain (#524 win
#2).
+    # The chunked upload already streams the whole-file md5 (`backend/api/uploads.py`), and
the
+    # submit hook records it on the job row (`submit_time_video_id`, one gs:// metadata stat
? #484);
+    # that value is BYTE-IDENTICAL to `compute_video_id` (same bytes, same MD5) and is
ALREADY
+    # trusted for the submit-time cache probe (`backend/api/jobs.py`). When True (default)
the drain
+    # reuses it and SKIPS re-hashing the multi-GB clip it just fetched to scratch; when the id
is absent
+    # (a local/appliance upload or a non-gcs direct-URI submit ? `submit_time_video_id`
returns None)
+    # the drain still hashes, exactly as before. Set False to force a re-hash unconditionally
+    # (belt-and-suspenders rollback ? reverts to pre-#524 behaviour with no redeploy).
+    trust_submit_video_id: bool = True

class BillingConfig(BaseModel):

```

9. assistant (2026-07-08T06:37:13.150Z)

Let me examine the upload path, submit flow, and how input refs are constructed to verify the reachability claims.

10. user (2026-07-08T06:37:16.120Z)

```

__init__.py
__pycache__
auth.py
job_worker.py
jobs.py
static
uploads.py
---uploads---
./backend/api/uploads.py

```

11. user (2026-07-08T06:37:16.337Z)

```

30:     db: Any, job_id: str, req: Any, owner_id: Optional[str], video_id: str = ""
38:     hashed until the worker runs, so the submitted ref is the stable pre-flight key.
    ``video_id`` is
51:         video_id=video_id,
92:         per video_id) and completes the job instantly: no drain, no GPU dispatch.
98:     video_id = orchestration.submit_time_video_id(input_ref, cfg) or ""
99:     probed = bool(video_id) and not req.force and orchestration.probe_process_cache(
100:         db, cfg, video_id
104:     outcome = claim_process_job(db, job_id, req, owner_id, video_id=video_id)
113:         db, cfg, video_id, req.video_path, requested_compute=req.compute
185:         "video_id": j.get("video_id"),

```

12. user (2026-07-08T06:37:19.808Z)

```

80     def submit_process_job(
81         db: Any, cfg: Any, job_id: str, req: Any, owner_id: Optional[str], input_ref: Any
82     ) -> Optional[Tuple[bool, JSONResponse]]:
83         """The whole /api/process submit decision (#448 dedup + #484 submit-time cache) in
one hook.
84
85         Order matters (#492 review):
86         1. Resolve a cheap md5 (gs:// metadata) and PROBE the cache ? read-only, no side
effects.
87         2. The remote-input scratch guard (507) runs ONLY on a probe miss: a cache hit never
88             fetches to local scratch, so low temp space must not reject it (review blocker
2).
89         3. CLAIM atomically (#481). A concurrent identical submit dedups here ? BEFORE any
90             re-hydrate side effect, so the first request owns the cache completion (blocker
1).
91         4. Only the claim WINNER realizes the hit (``cached_process_result`` ? itself
serialized
92             per video_id) and completes the job instantly: no drain, no GPU dispatch.
93
94         Returns ``(queued, response)``: ``queued`` True ? the caller kicks the drain;
``None`` ?
95         DB failure (the caller 500s)."""
96         from backend import orchestration # call-time seam: tests patch orchestration attrs
97
98         video_id = orchestration.submit_time_video_id(input_ref, cfg) or ""
99         probed = bool(video_id) and not req.force and orchestration.probe_process_cache(
100             db, cfg, video_id
101         )
102         if input_ref.backend != "local" and not probed:
103             _fetch_scratch_guard(cfg, input_ref) # raises 507 pre-claim, exactly as before
104         outcome = claim_process_job(db, job_id, req, owner_id, video_id=video_id)
105         if outcome is None:
106             return None
107         created, dedup_response = outcome
108         if not created:
109             assert dedup_response is not None # claim contract: not-created ? dedup
response
110             return (False, dedup_response)
111         if probed:
112             cached = orchestration.cached_process_result(
113                 db, cfg, video_id, req.video_path, requested_compute=req.compute
114             )
115             if cached is not None:
116                 db.mark_job_done(job_id, cached)
117                 return (
118                     False,
119                     JSONResponse(
120                         status_code=202,
121                         content={
122                             "job_id": job_id,
123                             "status": "done",

```

```

124         "poll": f"/api/jobs/{job_id}",
125         "cached": True,
126     },
127 ),
128 )
129 # The probe said hit but realizing it failed (report vanished / corrupt csv):
fall
130 # back to the normal queued dispatch ? but the scratch guard was skipped above,
so it
131 # must run NOW. If it fails, the already-claimed row is marked failed BEFORE
raising
132 # (a bare raise would leave a phantom 'queued' job for the drain to grab).
133 if input_ref.backend != "local":
134     try:
135         _fetch_scratch_guard(cfg, input_ref)
136     except HTTPException:
137         db.mark_job_failed(
138             job_id,
139             "cache re-hydrate failed and local scratch cannot hold a fetch of "

```

13. user (2026-07-08T06:37:20.243Z)

```

1     """Chunked upload (B2, PR-2) ? extracted from backend.main into its own router.
2
3     Browser uploads arrive in sequential parts ? security.max_part_mb (free-tier edge
4     proxies cap request bodies at ~100 MB ? HOSTING_PLAN $2). Where they assemble depends
5     on the input backend (CONTROL_PLANE_DURABLE_REBUILD P2):
6
7     * ``storage.input_backend == "local"`` (appliance/default): parts append to a partial
8     file under security.upload_dir, exactly as before.
9     * a remote input backend with an ``input_root`` (the hosted alpha: gcs): parts STREAM
10    through a ``storage.open_writer`` into ``<input_root>/uploads/<upload_id>/<name>``
11    while an incremental MD5 accumulates ? the assembled file never touches local
12    disk/tmpfs. The 4 GiB RAM-``/tmp`` control plane OOM'd on a 2.68 GB browser upload
13    (#480/#471); after this, instance memory bounds a CHUNK, not the video. ``complete``
14    returns the staged URI (inside the #465 input jail by construction ? it is under
15    ``input_root``) plus the streamed md5 as ``video_id``.
16
17    Upload state is in-process by design (this is the single-box alpha): a server restart
18    aborts partial uploads and the client re-inits. Per-user partitioning of upload_dir
19    arrives with PR-3 identity scoping.
20
21    The startup config lives on ``backend.main.global_config`` (set ONLY in ``main()``).
22    ``upload_cfg`` resolves it through the ``backend.main`` module object so it always
23    reads the live value the server set at startup (and so tests that monkeypatch
24    ``backend.main.global_config`` flow through unchanged).
25    """
26
27    import hashlib
28    import os
29    import threading
30    import unicodedata
31    import uuid
32    from typing import Any, Dict, Optional
33
34    from fastapi import APIRouter, HTTPException, Request
35    from pydantic import BaseModel
36    from starlette.concurrency import run_in_threadpool
37
38    from backend import storage
39    from backend.utils.app_home import (
40        resolve_output_dir, # P0a: app-home-aware path resolution
41    )
42    from backend.utils.freespace import require_free_bytes # HTTP-507 disk guard (P2, D11)
43    from backend.utils.hashing import compute_video_id # canonical file-identity hashing

```

```

44 from backend.utils.paths import safe_output_filename
45
46 router = APIRouter()
47
48 _uploads: Dict[str, Dict[str, Any]] = {}
49 _uploads_lock = threading.Lock()
50
51
52 class UploadInitRequest(BaseModel):
53     filename: str
54     total_size_bytes: Optional[int] = None
55
56
57 class UploadCompleteRequest(BaseModel):
58     total_parts: int
59
60
61 def _upload_cfg():
62     # Resolved lazily (import inside the function) to read the LIVE startup config that
63     # main() sets on the module global, and to avoid a circular import at module load
64     # (main.py imports this router; this module must not import main at import time).
65     import backend.main as _main
66
67     sec = getattr(_main.global_config, "security", None)
68     upload_dir = resolve_output_dir(
69         getattr(sec, "upload_dir", None) or "output/uploads"
70     ) # P0a: app-home-rooted (CWD-identical when RALLY_APP_HOME unset)
71     max_bytes = int(getattr(sec, "max_upload_mb", 4096) or 4096) * 1024 * 1024
72     part_bytes = int(getattr(sec, "max_part_mb", 95) or 95) * 1024 * 1024
73     exts = [
74         e.lower().lstrip(".")
75         for e in (getattr(sec, "allowed_upload_extensions", None) or ["mp4", "mov"])
76     ]
77     return upload_dir, max_bytes, part_bytes, exts
78
79
80 def upload_caps(cfg) -> Dict[str, int]:
81     """The upload limits /api/capabilities advertises (#480 item 5, CP-rebuild P5a).
82
83     ``max_upload_mb`` is what the SPA's pre-flight guard rejects oversize files against
84     ?
85     without it the guard is dead code and the user learns from a mid-upload 413.
86     ``max_part_mb`` is informational: each /api/upload/init response carries the
87     authoritative value the client actually chunks by."""
88     sec = getattr(cfg, "security", None)
89     return {
90         "max_upload_mb": int(getattr(sec, "max_upload_mb", 4096) or 4096),
91         "max_part_mb": int(getattr(sec, "max_part_mb", 95) or 95),
92     }
93
94 def _staging_target() -> "Optional[Dict[str, Any]]":
95     """The remote staging root + storage settings, or None for the local-file flow.
96
97     Remote staging is selected by the same input config /api/process resolves against
98     (storage.input_backend + input_root), so a staged upload is submittable verbatim.
99     """
100     import backend.main as _main
101
102     sc = getattr(_main.global_config, "storage", None)
103     if sc is None:
104         return None
105     input_backend = getattr(sc, "input_backend", "local") or "local"
106     input_root = (getattr(sc, "input_root", "") or "").rstrip("/")
107     if input_backend == "local" or not input_root:

```

```

108         return None
109     cfg = sc.model_dump() if hasattr(sc, "model_dump") else dict(sc)
110     return {"root": input_root, "config": cfg}
111
112
113 def _abort_upload(upload_id: str) -> None:
114     with _uploads_lock:
115         st = _uploads.pop(upload_id, None)
116     if not st:
117         return
118     writer = st.get("writer")
119     if writer is not None:
120         # A streaming writer has no cancel ? close() finalizes whatever was staged, then
121         # the partial object is deleted. Both are best-effort: the upload is already
122         # aborted from the client's point of view.
123         try:
124             writer.close()
125         except Exception: # noqa: BLE001 ? abort cleanup must never mask the abort
126             pass
127         try:
128             ref = storage.parse_uri(st["stage_uri"])
129             storage.get_storage(ref.backend, config=st.get("config")).delete(ref)
130         except Exception: # noqa: BLE001
131             pass
132     return
133     try:
134         os.remove(st["tmp_path"])
135     except OSError:
136         pass
137
138
139 def _normalize_upload_filename(filename: str) -> str:
140     """Normalize browser-provided file names before extension validation.
141
142     Some mobile/file-provider pickers can hand us visually normal names with
143     compatibility
144     characters (for example a fullwidth dot) or trailing whitespace/dots. Keep the
145     existing
146     traversal/null-byte checks in ``safe_output_filename`` as the authority after this
147     cleanup.
148     """
149     return unicodedata.normalize("NFKC", filename).strip().rstrip(". ")
150
151
152 @router.post("/api/upload/init")
153 def upload_init(req: UploadInitRequest):
154     """Start a chunked upload. Returns upload_id; PUT sequential parts next."""
155     upload_dir, max_bytes, part_bytes, exts = _upload_cfg()
156     try:
157         name = safe_output_filename(_normalize_upload_filename(req.filename))
158     except ValueError as e:
159         raise HTTPException(status_code=400, detail=str(e)) from e
160     ext = os.path.splitext(name)[1].lower().lstrip(".")
161     if ext not in exts:
162         raise HTTPException(
163             status_code=400,
164             detail=f"Extension '{ext}' not allowed. Allowed: {sorted(exts)}",
165         )
166     if req.total_size_bytes is not None and req.total_size_bytes > max_bytes:
167         raise HTTPException(
168             status_code=413,
169             detail=f"File exceeds the {max_bytes // (1024 * 1024)} MB upload limit.",
170         )
171     upload_id = uuid.uuid4().hex
172     staging = _staging_target()

```

```

170     if staging is not None:
171         # Remote staging: stream parts straight into the bucket under the input root ?
172         # never assembled on local disk/tmpfs (CONTROL_PLANE_DURABLE_REBUILD P2). The
173         # upload_id in the key keeps concurrent same-name uploads distinct, and the
free-
174         # space guard is skipped: no local bytes are consumed.
175         stage_uri = f"{staging['root']}/uploads/{upload_id}/{name}"
176         try:
177             writer = storage.open_writer(stage_uri, config=staging["config"])
178         except NotImplementedError:
179             staging = None # backend can't stream (e.g. mounted-Drive) ? stage locally
180         else:
181             with _uploads_lock:
182                 _uploads[upload_id] = {
183                     "filename": name,
184                     "writer": writer,
185                     "hasher": hashlib.md5(),
186                     "stage_uri": stage_uri,
187                     "config": staging["config"],
188                     "parts": 0,
189                     "bytes": 0,
190                 }
191             return {
192                 "upload_id": upload_id,
193                 "max_part_mb": part_bytes // (1024 * 1024),
194                 "next_part": f"/api/upload/{upload_id}/part/0",
195             }
196     os.makedirs(upload_dir, exist_ok=True)
197     # P2 (D11): fail fast with 507 if the box can't hold the declared upload. Best-
effort ? an
198     # undeclared size (total_size_bytes=None) or an unreadable disk skips the guard
(never blocks a
199     # legit upload); the per-part total cap in upload_part is the hard backstop either
way.
200     require_free_bytes(upload_dir, req.total_size_bytes, stage="upload")
201     tmp_path = os.path.join(upload_dir, f".partial-{upload_id}")
202     open(tmp_path, "wb").close()
203     with _uploads_lock:
204         _uploads[upload_id] = {
205             "filename": name,
206             "tmp_path": tmp_path,
207             "parts": 0,
208             "bytes": 0,
209         }
210     return {
211         "upload_id": upload_id,
212         "max_part_mb": part_bytes // (1024 * 1024),
213         "next_part": f"/api/upload/{upload_id}/part/0",
214     }
215
216
217 @router.put("/api/upload/{upload_id}/part/{index}")
218 async def upload_part(upload_id: str, index: int, request: Request):
219     """Append one part (raw bytes body). Parts are strictly sequential (0,1,2,...)."""
220     _, max_bytes, part_bytes, _ = _upload_cfg()
221     with _uploads_lock:
222         st = _uploads.get(upload_id)
223     if st is None:
224         raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
225     if index != st["parts"]:
226         raise HTTPException(
227             status_code=409,
228             detail=f"Out-of-order part: expected {st['parts']}, got {index} (parts are
sequential)",
229         )

```

```

230
231 received = 0
232 overflow = None
233 if st.get("writer") is not None:
234     # Remote staging: buffer THIS part (? max_part_mb ? the caps above still abort
235     an
236     # oversize stream), then hand it to the blocking bucket writer in one threadpool
237     # hop so a network flush never stalls the event loop mid-upload.
238     buf = bytearray()
239     async for chunk in request.stream():
240         received += len(chunk)
241         if received > part_bytes:
242             overflow = (
243                 f"Part exceeds the {part_bytes // (1024 * 1024)} MB per-part limit."
244             )
245             break
246         if st["bytes"] + received > max_bytes:
247             overflow = (
248                 f"Upload exceeds the {max_bytes // (1024 * 1024)} MB total limit."
249             )
250             break
251         buf += chunk
252     if not overflow and buf:
253         data = bytes(buf)
254         await run_in_threadpool(st["writer"].write, data)
255         st["hasher"].update(data)
256     else:
257         with open(st["tmp_path"], "ab") as fh:
258             async for chunk in request.stream():
259                 received += len(chunk)
260                 if received > part_bytes:
261                     overflow = (
262                         f"Part exceeds the {part_bytes // (1024 * 1024)} MB per-part
263                         limit."
264                     )
265                     break
266                 if st["bytes"] + received > max_bytes:
267                     overflow = (
268                         f"Upload exceeds the {max_bytes // (1024 * 1024)} MB total
269                         limit."
270                     )
271                     break
272                 fh.write(chunk)
273     if overflow:
274         _abort_upload(upload_id)
275         raise HTTPException(
276             status_code=413, detail=overflow + " Upload aborted ? re-init to retry."
277         )
278
279     with _uploads_lock:
280         st["parts"] += 1
281         st["bytes"] += received
282     return {
283         "upload_id": upload_id,
284         "received_parts": st["parts"],
285         "received_bytes": st["bytes"],
286     }
287
288 @router.post("/api/upload/{upload_id}/complete")
289 def upload_complete(upload_id: str, req: UploadCompleteRequest):
290     """Finalize: verify part count, move into place, return {path, video_id} for
291     /api/process."""
292     upload_dir, _, _, _ = _upload_cfg()
293     with _uploads_lock:

```

```

291         st = _uploads.get(upload_id)
292     if st is None:
293         raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
294     if st["parts"] == 0 or st["parts"] != req.total_parts:
295         raise HTTPException(
296             status_code=409,
297             detail=f"Expected {req.total_parts} part(s), received {st['parts']}.",
298         )
299     if st.get("writer") is not None:
300         # Remote staging: close() finalizes the streamed object; the md5 accumulated per
301         # part IS compute_video_id's digest (same bytes, same algorithm), so
/api/process
302         # gets the identity without anyone re-reading the video. (Sync endpoint ?
FastAPI
303         # runs it on the threadpool, so the blocking finalize doesn't stall the loop.)
304         try:
305             st["writer"].close()
306         except Exception as e: # noqa: BLE001 ? surface a clean verdict, then clean up
307             _abort_upload(upload_id)
308             raise HTTPException(
309                 status_code=502,
310                 detail=f"Failed to finalize the staged upload ({e}). Re-init to retry.",
311             ) from e
312         with _uploads_lock:
313             _uploads.pop(upload_id, None)
314         return {
315             "path": st["stage_uri"],
316             "video_id": st["hasher"].hexdigest(),
317             "size_bytes": st["bytes"],
318             "next": "POST /api/process with this path",
319         }
320     final_path = os.path.join(upload_dir, st["filename"])
321     if os.path.exists(
322         final_path
323     ): # never overwrite an earlier upload with the same name
324         stem, ext = os.path.splitext(st["filename"])
325         final_path = os.path.join(upload_dir, f"{stem}-{upload_id[:8]}{ext}")
326     os.replace(st["tmp_path"], final_path)
327     with _uploads_lock:
328         _uploads.pop(upload_id, None)
329     video_id = compute_video_id(final_path)
330     return {
331         "path": os.path.abspath(final_path),
332         "video_id": video_id,
333         "size_bytes": os.path.getsize(final_path),
334         "next": "POST /api/process with this path",
335     }
336
337
338 @router.delete("/api/upload/{upload_id}")
339 def upload_abort(upload_id: str):
340     """Abort an in-flight upload and delete its partial file."""
341     with _uploads_lock:
342         known = upload_id in _uploads
343     if not known:
344         raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
345     _abort_upload(upload_id)
346     return {"status": "aborted", "upload_id": upload_id}
347

```

14. assistant (2026-07-08T06:37:59.030Z)

Let me verify the direct-URI input jail and confirm the fetch path has no generation pinning.

15. user (2026-07-08T06:38:03.067Z)

Found 8 files

```
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\backend\orchestration.py
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\backend\config\models.py
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\backend\utils\paths.py
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\backend\storage\ref.py
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\backend\storage\capabilities.py
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\backend\storage\__init__.py
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\backend\main.py
C:\Users\avidu\AppData\Local\Temp\claude\wt524-969\backend\api\uploads.py
```

16. user (2026-07-08T06:38:03.216Z)

No matches found

17. user (2026-07-08T06:38:09.996Z)

```
554: def _source_cache_path(video_id: str, input_ref, cfg) -> str:
563: def _cached_remote_source(video_id: str, input_ref, cfg) -> str:
747:     hosted alpha jails inputs and has no per-user medium credentials, so it stays upload-to-
local.
924:     # In hosted mode a bucket URI is allowed only when it's under the configured input_root
(the
927:         getattr(request.app.state.config, "storage", None), "input_root", None
1236:     auth on without the ``allowed_video_root`` jail / CF team+AUD). A strict no-op when no
```

18. user (2026-07-08T06:38:12.536Z)

```
915     # spoofed identity header fails here with 401, before any work) and tags the job's
owner_id.
916     try:
917         owner_id = edge_auth.resolve_tenant(request.headers, request.app.state.config)
918     except edge_auth.EdgeAuthError as e:
919         raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e
920
921     allowed_root = getattr(
922         getattr(request.app.state.config, "security", None), "allowed_video_root", None
923     )
924     # In hosted mode a bucket URI is allowed only when it's under the configured
input_root (the
925     # designated bucket) ? not an arbitrary bucket the runtime SA happens to reach
(tenant isolation).
926     allowed_uri_root = getattr(
927         getattr(request.app.state.config, "storage", None), "input_root", None
928     )
929     try:
930         validate_input_path(
931             req.video_path, allowed_root=allowed_root, allowed_uri_root=allowed_uri_root
932         )
933     # M2: resolve the input once (path or storage URI) ? raises ValueError for an
unsupported
934     # scheme, which we map to a clean 400 (an unknown scheme must never 500).
935     input_ref = storage.resolve_input_ref(
936         req.video_path, getattr(request.app.state.config, "storage", None)
937     )
938     except ValueError as e:
939         raise HTTPException(status_code=400, detail=str(e)) from e
940     # The local-existence fast-fail applies only to a LOCAL input, and stats the
RESOLVED local path
941     # (input_ref.key ? e.g. local://? stripped to the bare path), not the raw URI
string. A
942     # bucket/Drive URI can't be cheaply os.path.exists()'d ? its existence is verified
when the worker
943     # fetches it (a fetch miss fails the job loudly). validate_input_path above still
runs for every
```

```

944      # input: its null-byte guard always applies, and a configured allowed_video_root
(hosted mode)
945      # rejects a non-local URI as out-of-root.
946      if input_ref.backend == "local" and not os.path.exists(input_ref.key):
947          raise HTTPException(
948              status_code=404, detail=f"Video file not found at '{req.video_path}'"
949          )
950      # The P2 (D11) remote-source scratch guard now lives INSIDE jobs.submit_process_job,
AFTER
951      # the cache probe (#492 review): a cached gs:// result never fetches to local
scratch, so
952      # low temp space must not 507 it; a cache MISS still fails fast pre-claim exactly as
before.
953      if req.segmenter and not segmenter_registry.get(req.segmenter):
954          available = list(segmenter_registry.list_available().keys())
955          raise HTTPException(
956              status_code=400,
957              detail=f"Segmenter '{req.segmenter}' not found. Available segmenters:
{available}",
958          )
959
960      # ATOMIC idempotent submit (#448) + submit-time md5 cache (#484): one hook claims-
or-dedups,
961      # and completes the job INSTANTLY when this video's detection already exists in the
DB or the
962      # bucket (no drain, no GPU dispatch). `force` opts out. Logic in backend.api.jobs.
963      job_id = uuid.uuid4().hex
964      outcome = jobs_api.submit_process_job(

```

19. assistant (2026-07-08T06:38:33.002Z)

I have verified the full trust path and the reachability boundary. Let me record my verdict.

The mechanic the finding describes is real in the code: the trusted branch (orchestration.py:791-796) takes `video\_id = candidate` with no post-fetch recheck, and `GCSStorage.fetch\_to\_local` (gcs.py:65) calls `download\_to\_filename(dst)` with no `if\_generation\_match` (grep for `if\_generation\_match`/`generation` across `backend/storage` returns zero matches). So `download\_to\_filename` verifies fetched bytes against the object's *current* md5Hash, not the submit-time `X`. Pre-#524 the drain re-hashed (compute_video_id), which was self-correcting.

But the finding is REFUTED as a must-fix defect because the required precondition ? the gs:// object being overwritten with different-hash content between the submit-time stat and the drain fetch ? is not constructible in the actual current code:

- Uploads are server-mediated and stream to a fresh `uploads/<uuid4>/<name>` key (uploads.py:168,175); clients hold no direct GCS write creds, and a re-upload gets a new uuid path, so no client can overwrite a submitted object.
- Direct-URI `/api/process` submits are jailed to `storage.input\_root` for READ only (main.py:924-932, `validate\_input\_path(allowed\_uri\_root=...)`); the jail grants no write path.
- The server itself never overwrites an existing input object ? uploads land on fresh uuid keys, outputs live under a different `outputs/<md5>/` prefix.
- `jobs.video\_id` is server-derived from `submit\_time\_video\_id` (jobs.py:98), never a client-supplied field, and the object is strongly consistent and immutable post-`writer.close()`, so `X` always equals the bytes the drain fetches.

The reviewer's own text concedes this ("not client-triggerable in the current hosted config? defense-in-depth today? becomes client-reachable the moment any future presigned direct-to-bucket upload? lands" ? a direction the design doc explicitly REJECTS). No concrete current input yields the poisoned-cache outcome; it is unreachable hardening, not a defect a maintainer must fix now.

20. user (2026-07-08T06:38:46.566Z)

Structured output provided successfully